Validation
Estimating the performance of the model on unseen data

- Split into training and test set
- k-fold cross validation
- leave-one-out cross validation

Split into training and test set

- Randomly split data set into a training set and a test set.
  Usually 80% or 90% of data used for training, remaining data for testing

- Stratified split
  Same as above, but ensure that the proportion of classes are the same in the training and test set (first divide data by class, the split each subset)

Split into training and test set

Advantage:

- Fast - Model needs to be trained only once

Disadvantages:

- Varying results from run to run, as test set is relatively small
- Slightly pessimistic estimation, as we use most, but not all available data for training

K-fold Cross Validation

- Randomly split data set into a k groups (or folds) of equal size

```
For i in range(k):
   Use groups 0,...,i-1,i+1,...,k-1 for training
   Use group i for testing
```

  Compute performance metrics for all test cases

- Stratified k-fold – Same as above, but ensure that the proportion of classes is the same in each of the groups

K-fold Cross Validation

Advantage:

- Little variation between runs – all data are used for testing exactly once (and for training k-1 times)

Disadvantages:

- Time-consuming – model needs to be trained k times.
- Slightly pessimistic estimation, as we use most, but not all available data for training

Leave-one-out cross validation

- Same as k-fold cross validation with k = size of the data set

```
For i in range(data set size):
   Use examples 0,...,i-1,i+1,...,k-1 for training
   Use examples i for testing
```

Leave-one-out cross validation

Advantage:

- Little or no variation between runs – depending on whether the learning algorithm is stochastic).
- Most realistic estimation, since the training set is virtually the same as the whole dataset.

Disadvantages:

- Extremely time-consuming – model needs to be trained once for every training example.

Leave-one-out cross validation

Normally we use leave-one-out cross validation only in these situations:

1. When the available dataset is very small

2. When the leave-one-out process can be integrated into the algorithm
   For instance, in 1-nearest neighbor:
   We find the distance from every example to every other example in the dataset
   For every example, we predict as its class the class of the **second** most similar example in the dataset